

Practical Next Milestone Directions for StratLake Trade Engine

Repository baseline

StratLake's current center of gravity is clear: it is an artifact-first, deterministic quantitative research platform built for research review rather than ingestion or live trading, and it explicitly treats notebooks, APIs, CLI entrypoints, and pipeline/orchestrator wrappers as multiple entry points into one execution system rather than separate workflow stacks. The stable `src.execution` surface exports strategy, alpha, portfolio, pipeline, campaign, benchmark-pack, and validation entrypoints, while the notebook and pipeline integration docs repeatedly state the same non-duplication principle. [1](#) [2](#) [3](#) [4](#) [5](#)

The repository structure also matches that identity. At the root, the repo exposes `.github`, `artifacts`, `cli`, `configs`, `contracts`, `docs`, `examples`, `src`, and `tests`. Inside `src/`, there are first-class `catalog`, `execution`, `pipeline`, `portfolio`, `research`, and `validation` packages, and `src/research/` now includes both `robustness.py` and a broad set of research/governance-related modules. The repo also carries dedicated GitHub Actions workflows for CI, milestone validation, and release publication. [6](#) [7](#) [8](#) [9](#)

The recent milestone history matters a great deal for M35. README and milestone docs show that M29 already introduced a read-only unified research catalog with indexer, lineage extraction, query APIs, validation, and notebook workflows; M31 established the canonical promotion policy path; M32 added governance observability without replaying policy; M33 hardened cross-platform CI, packaging, and release posture; and M34 added a deterministic statistical robustness evidence layer that is explicitly review context rather than silent decision enforcement. [1](#) [10](#) [11](#) [12](#) [13](#) [14](#) [15](#) [16](#) [17](#) [18](#) [19](#)

That leads to the single most important planning conclusion for M35: your instinct toward lineage, discovery, and robustness-aware cataloging is directionally strong, but it should be framed as an **extension of the existing M29 catalog/query/lineage stack**, not as a new catalog milestone from scratch. The repository already has `src/catalog`, a public `src.catalog` API, a query CLI, lineage extraction, and validation; re-proposing “catalog + lineage + discovery” as if they do not exist would understate the repository's current capabilities and create avoidable duplication risk. [10](#) [11](#) [12](#) [13](#) [14](#) [20](#)

What should follow M34

The most natural question after M34 is not “how do we add another execution engine or another policy engine,” but “how do we make all this new robustness evidence easier to discover, relate, compare, and review?” M34 already writes deterministic robustness bundles with summary, findings, WFE, sample-size, sensitivity, multiple-testing, and temporal-validation artifacts; M32 can already surface robustness fields in governance outputs; and both milestones are explicit that governance remains observational unless a future policy layer is intentionally added. In other words, the repository now has more evidence than

before, and the highest-value next step is better evidence navigation, not silent new enforcement. 18 19

16 21

That reading also lines up with how adjacent platforms are typically structured. MLflow Tracking centers experiments around runs, models, parameters, metrics, and artifacts, and exposes search over runs and logged models. OpenLineage, meanwhile, defines an open lineage model around datasets, jobs, and runs with extensible metadata facets. Those patterns point toward a practical next milestone for StratLake: deepen evidence discovery and relationship surfacing around existing artifacts rather than invent a second canonical registry or a second execution layer. StratLake is already closer to that architecture than to a greenfield monitoring platform. 22 10 11 12

By contrast, several other candidate directions are real but heavier. Monitoring systems are their own observability domain: Evidently documents monitoring as a distinct surface for data quality and drift, not just artifact browsing. Likewise, Apache Iceberg's schema-evolution docs show how quickly "dataset contract evolution" becomes a substantive metadata subsystem with add/drop/rename/update/reorder semantics and correctness guarantees. Those are valuable directions, but they are larger, more infrastructure-heavy, and less immediately adjacent to M34 than a robustness-aware discovery and evidence-navigation layer.

23

Candidate directions assessed

Robustness-aware research catalog and experiment lineage browser. Milestone title: M35 — Robustness-Aware Research Catalog Extensions. Description: extend the existing M29 catalog/indexer/query/lineage stack so robustness bundles, governance reports, milestone-validation bundles, and release artifacts become first-class catalog and lineage objects, with richer browsing and review flows. Why it follows naturally from M34: M34 added a large new evidence surface, and M32 already knows how to expose some of it, so the next practical value is making that evidence easy to locate and compare across runs. Architectural fit: excellent, because it can remain read-only, artifact-driven, deterministic, and entripoint-neutral. Risks: the main risk is accidentally rebuilding M29 from scratch or slipping into a database/dashboard project. Dependencies: M29 catalog package, M32 governance outputs, M34 robustness bundle, portable path rules, existing release validation posture. Estimated effort: medium. Likely GitHub issues: 5–7. Validation strategy: deterministic query snapshots, lineage tests, empty-artifact smoke, local explorer smoke, cross-platform path checks, CI-safe examples. Examples/docs impact: meaningful but platform-first, not docs-only. Interface fit: notebook-friendly, pipeline-friendly, API-friendly, CLI-friendly, and CI-friendly. This is the strongest single candidate, but only if it is explicitly scoped as an **M29 extension**. 10

11 12 13 16 18

Artifact discovery/search layer over strategy, alpha, portfolio, campaign, governance, and robustness outputs. Milestone title: M35 — Unified Artifact Discovery. Description: add richer search facets, artifact-type indexing, related-run lookup, and evidence-oriented filtering over the current catalog instead of forcing users to know artifact roots up front. Why it follows naturally from M34: robustness outputs are only valuable when researchers can actually find them across many runs. Architectural fit: very strong, because M29 already has deterministic catalog records and query helpers, and M34/M32 add new evidence dimensions to index. Risks: as a standalone milestone it is too narrow and overlaps substantially with existing M29 query capabilities. Dependencies: src/catalog, existing query CLI/API, M34 bundle metadata, governance outcome fields. Estimated effort: small-to-medium. Likely GitHub issues: 3–5. Validation strategy: query/filter snapshots, deterministic ordering checks, zero-record and sparse-root smoke tests,

example scripts. *Examples/docs impact*: moderate. *Interface fit*: all five interfaces are strong. This is best treated as a **core issue group inside the preferred milestone**, not as a full standalone M35. 13 15 16

18

Cross-run provenance graph connecting features, strategies, alpha models, portfolios, campaigns, robustness reports, and governance outcomes. *Milestone title*: M35 – Extended Provenance Graph. *Description*: push lineage from current run-to-run relationships toward richer evidence relationships, especially robustness/governance/release linkage and, potentially, feature/dataset ancestry. *Why it follows naturally from M34*: robustness and governance now sit adjacent to core runs, so evidence linkage has become more valuable. *Architectural fit*: medium-to-strong if scoped to read-only derived edges; weaker if it tries to introduce persistent graph storage or full dataset lineage. *Risks*: M29 already shipped lineage extraction, and its design explicitly kept feature dataset builds out of catalog scope; widening the boundary to features/data contracts is real platform work but materially larger. *Dependencies*: existing lineage layer, manifest/reference metadata, robustness report paths, campaign/scenario metadata. *Estimated effort*: medium-to-large. *Likely GitHub issues*: 5–8. *Validation strategy*: edge determinism tests, orphan-reference behavior tests, robustness/governance/release relationship fixtures. *Examples/docs impact*: moderate. *Interface fit*: notebook/API/CLI strong; pipeline/CI good. Best considered either a **sub-scope inside the preferred milestone** or an M36 follow-on if feature datasets are included. 10 12 16 18

Robustness-aware promotion policy simulation, clearly separated from actual promotion decisions.

Milestone title: M35 – Promotion Policy Simulation Sandbox. *Description*: simulate hypothetical policy outcomes if robustness evidence were treated as policy inputs, while preserving canonical `promotion_gates.json` outputs and keeping simulated decisions clearly non-canonical. *Why it follows naturally from M34*: M34's core principle is that robustness should make research risk visible without silently changing promotion decisions, and simulation is the cleanest way to study “what would have happened” without crossing that boundary. *Architectural fit*: good if implemented as read-only derived artifacts. *Risks*: the naming, docs, and artifact schemas would need to be extremely explicit so users never confuse simulated outcomes with actual promotion results. *Dependencies*: M31 canonical policy path, M32 governance observability, M34 robustness status/reason codes. *Estimated effort*: medium. *Likely GitHub issues*: 4–6. *Validation strategy*: fixture-based hypothetical policy cases, canonical-versus-simulated separation tests, regression checks that no recorded policy artifacts change. *Examples/docs impact*: meaningful. *Interface fit*: notebook/API/CLI/CI strong; pipeline fit is still good. This is a strong later candidate, but narrower than the catalog/discovery path. 21 16 18

Model/risk monitoring layer for alpha and portfolio outputs. *Milestone title*: M35 – Alpha and Portfolio Monitoring Signals. *Description*: define post-run monitoring artifacts over alpha and portfolio outputs, such as rolling degradation summaries, threshold breaches, and exposure/turnover/risk-health drift. *Why it follows naturally from M34*: once robustness exists, the next question is often whether behavior degrades across later runs or later data windows. *Architectural fit*: acceptable if it remains artifact-first and retrospective; weaker if it drifts into always-on operational monitoring. *Risks*: monitoring typically wants baselines, current-versus-reference comparisons, scheduling, and sometimes alerting, which pushes toward a broader observability system. *Dependencies*: alpha/portfolio output contracts, rolling metrics, possibly dataset/time-window conventions. *Estimated effort*: medium-to-large. *Likely GitHub issues*: 6–9. *Validation strategy*: synthetic drift fixtures, monotonic time-window tests, threshold summary snapshot tests. *Examples/docs impact*: significant. *Interface fit*: notebook/API/CLI/CI good; pipeline fit moderate-to-strong. This is valuable, but it is less immediately aligned with the current repo identity than a stronger evidence-discovery milestone. 1 24

Regime-aware drift monitoring and robustness dashboards. *Milestone title:* M35 – Regime Drift Observatory. *Description:* add regime-conditioned drift summaries and visual dashboards that relate robustness evidence to regime changes and changing behavior across windows. *Why it follows naturally from M34:* M24–M27 already built regime-aware research surfaces, and M34 added robustness diagnostics; combining them is intellectually natural. *Architectural fit:* mixed. A read-only local report surface fits; a dashboard platform does not fit as well because M29 and M32 explicitly avoided dashboard/database backends. *Risks:* scope creep toward a visualization product or service. *Dependencies:* regime metadata, robustness bundles, longitudinal comparison logic, local rendering conventions. *Estimated effort:* large. *Likely GitHub issues:* 7–10. *Validation strategy:* static report fixtures, deterministic chart/table rendering, no-service local explorer tests. *Examples/docs impact:* high. *Interface fit:* notebook and local CLI strong; pipeline/API/CI okay if rendered as static artifacts. Good future work, but too broad for the best M35. 1 10 16

18

Data-quality observability dashboards over feature and market datasets. *Milestone title:* M35 – Dataset Quality Observatory. *Description:* build artifactized data-quality summaries and visual observability over input datasets rather than only over strategy/portfolio outputs. *Why it follows naturally from M34:* thin or biased evidence is often upstream-data driven. *Architectural fit:* medium, but it pushes on a boundary that M29 explicitly treated as out of scope: feature dataset builds are listed as input surfaces rather than research experiment outputs. *Risks:* this changes the platform boundary from “research artifact observability” toward “data platform observability,” which is a larger strategic move. *Dependencies:* canonical dataset contracts, feature-build artifact standards, quality metric definitions, likely data profiling. *Estimated effort:* large. *Likely GitHub issues:* 7–10. *Validation strategy:* dataset fixture profiles, schema drift fixtures, path portability tests. *Examples/docs impact:* high. *Interface fit:* notebook/API/CLI/CI fine; pipeline fit good. This is real platform work, but it should probably come **after** stronger dataset contracts exist. 10

Dataset contract evolution and schema migration support. *Milestone title:* M35 – Dataset Contract Evolution. *Description:* define versioned dataset contracts, compatibility checks, and migration helpers for feature/market datasets. *Why it follows naturally from M34:* robustness and repeatability become more trustworthy when upstream datasets are versioned and schema-safe. *Architectural fit:* medium, because it is more data-platform work than research-surface work. *Risks:* schema evolution is a serious subsystem; even mature table formats such as Apache Iceberg treat add/drop/rename/update/reorder operations and correctness guarantees as core platform features, not as lightweight niceties. *Dependencies:* contract schemas, migration policy, dataset versioning, backward compatibility rules, possibly storage-format conventions. *Estimated effort:* large-to-extra-large. *Likely GitHub issues:* 8–12. *Validation strategy:* contract compatibility fixtures, migration round-trip tests, deterministic upgrade/downgrade reports. *Examples/docs impact:* high. *Interface fit:* API/CLI/pipeline/CI strong; notebook fit secondary. Important long-term, but too large for the best next milestone unless you intentionally pivot toward data-platform infrastructure. 10

25

Incremental cache invalidation and artifact dependency graph. *Milestone title:* M35 – Artifact Dependency and Cache Graph. *Description:* create derived-state caching and invalidation semantics across artifacts so notebooks, CLIs, and pipelines can avoid rescanning unchanged roots. *Why it follows naturally from M34:* expanding artifact families make full scans costlier and increase the appeal of faster discovery. *Architectural fit:* weaker than it may first appear, because M29 explicitly preserved an on-demand, read-only, in-memory model and rejected persistent cache as a canonical layer. *Risks:* high architectural risk. Invalidated caches can become a second source of truth unless the semantics are extremely disciplined. *Dependencies:* dependency graph semantics, cache storage, change detection, versioning of derived

snapshots. *Estimated effort*: large. *Likely GitHub issues*: 6–10. *Validation strategy*: stale-cache detection, incremental rebuild fixtures, canonical-versus-derived parity tests. *Examples/docs impact*: moderate. *Interface fit*: pipeline/API/CLI/CI strong if done well; notebook fit also good. This is meaningful infrastructure work, but it is not the best immediate M35 because it adds statefulness where the current stack is intentionally read-only and on-demand. 10 11 13

Notebook-first research UX improvements using existing execution APIs. *Milestone title*: M35 – Notebook Research Ergonomics. *Description*: tighten notebook helpers, catalog convenience functions, and exploration examples using existing src.execution and src.catalog APIs. *Why it follows naturally from M34*: richer evidence is only helpful if notebook-driven researchers can inspect it quickly. *Architectural fit*: strong, because notebook docs already insist on thin wrappers over shared execution/catalog contracts. *Risks*: as a whole milestone, this risks being more UX/documentation than platform improvement. *Dependencies*: existing execution result APIs, catalog query helpers, examples. *Estimated effort*: small-to-medium. *Likely GitHub issues*: 3–5. *Validation strategy*: script-backed notebook examples, import safety, empty-root smoke tests, cross-layer artifact parity checks. *Examples/docs impact*: very high. *Interface fit*: notebook strongest, API/CLI/CI still good, pipeline less central. This is an excellent **supporting issue stream**, but probably not a high-value standalone M35 right after M34. 2 15 4

Lightweight local report explorer for artifacts. *Milestone title*: M35 – Local Artifact Report Explorer. *Description*: add a read-only local explorer that renders catalog rows, artifact inventories, robustness bundles, and governance summaries into static table/Markdown/HTML views without a database or service. *Why it follows naturally from M34*: robustness evidence is easier to review when users do not have to manually open many JSON and CSV files. *Architectural fit*: strong if it remains local, static, and derived from existing artifacts. *Risks*: if it becomes a dashboard server or persistent app, it stops being a good fit for current architecture. *Dependencies*: M29 catalog/query APIs, M32 governance outputs, M34 bundles, portable path rules. *Estimated effort*: medium. *Likely GitHub issues*: 4–6. *Validation strategy*: static render fixtures, deterministic output tests, example bundle smoke, docs/path lint. *Examples/docs impact*: high. *Interface fit*: notebook/API/CLI/CI strong; pipeline good via generated static reports. This is **very attractive as part of the preferred milestone**, but slightly thin as a standalone milestone unless bundled with discovery and lineage extensions. 10 13 15 16 18

Release-readiness index across milestone validation bundles. *Milestone title*: M35 – Release Readiness Index. *Description*: aggregate milestone validation, docs/path lint, deterministic rerun, packaging/build, release checks, and milestone-specific checklists into a single readiness artifact. *Why it follows naturally from M34*: M33 and M34 both emphasize release and validation posture, and a compact readiness index would make those signals more navigable. *Architectural fit*: good, because it can remain a derived, read-only summary over existing validation outputs. *Risks*: it is probably too narrow to carry the whole milestone. *Dependencies*: milestone validation bundle, CI workflow outputs, release workflow, M34 checklist conventions. *Estimated effort*: small-to-medium. *Likely GitHub issues*: 2–4. *Validation strategy*: fixture bundles, score aggregation snapshots, missing-artifact degradation tests. *Examples/docs impact*: moderate. *Interface fit*: notebook/API/CLI/CI strong; pipeline fit moderate. Best treated as a **feature inside an audit/discovery milestone**, not as the milestone itself. 7 8 9 26

Enhanced governance audit trail linking promotion decisions, robustness evidence, and release history. *Milestone title*: M35 – Governance Audit Trail and Release Linkage. *Description*: extend M32 governance observability so promotion outcomes, robustness evidence, release bundles, milestone validation artifacts, and possibly tag/release notes are linked into one read-only audit trail. *Why it follows*

naturally from M34: M32 already surfaces promotion governance, and M34 already adds robustness context; the missing piece is stronger linkage to release/milestone validation history. *Architectural fit*: very strong, because it remains observational and read-only. *Risks*: narrower day-to-day utility for researchers than catalog/discovery improvements, and some users may view it as “ops-first.” *Dependencies*: M32 outcome matrix architecture, M34 robustness fields, release workflow artifacts, milestone validation bundle. *Estimated effort*: medium. *Likely GitHub issues*: 4–6. *Validation strategy*: end-to-end audit fixtures, missing-evidence degradation tests, canonical-versus-derived separation checks. *Examples/docs impact*: meaningful. *Interface fit*: notebook/API/CLI/CI strong; pipeline good. This is the **best alternative** if you want M35 to lean more toward operational maturity than research discovery. 16 18 19 9 8

Production runtime readiness: config profiles, environment validation, secrets hygiene, and reproducible runtime bundles. *Milestone title*: M35 – Runtime Readiness Profiles. *Description*: add environment validation, secret-safe config conventions, reproducible runtime profile bundles, and deployment-adjacent hygiene around execution environments. *Why it follows naturally from M34*: it fits the user goal of improving production readiness and release operational maturity. *Architectural fit*: mixed. It can preserve artifact determinism, but it nudges StratLake toward a deployment/runtime toolchain while the README still frames the platform as research-review oriented rather than live-trading oriented. *Risks*: strategic drift away from the repo’s current identity, plus added complexity around secrets and environment management. *Dependencies*: packaging/build posture, config/profile conventions, environment checks, release workflow. *Estimated effort*: medium-to-large. *Likely GitHub issues*: 5–8. *Validation strategy*: env-validation fixtures, secrets-redaction tests, portable runtime-bundle smoke tests, release workflow checks. *Examples/docs impact*: high. *Interface fit*: API/CLI/pipeline/CI strong; notebook moderate. This is a legitimate later option, but it is less natural than a discovery/evidence milestone immediately after M34. 1 17 9

Which ideas are platform work and which are not

The clearest **real platform improvements** for the next milestone are the ones that deepen existing artifact contracts without adding a new source of truth: robustness-aware catalog extensions, richer artifact discovery, extended evidence lineage, a lightweight local explorer, and a governance/release audit trail. Those all build directly on current M29, M32, M33, and M34 foundations and preserve the repository’s declarative, deterministic, read-only-observability boundaries. 10 16 18 17

The clearest **documentation-heavy or UX-heavy improvements** are notebook-first research ergonomics and some local explorer polish when taken by themselves. Those are worth doing, but the repo already has notebook execution integration, catalog notebook workflows, and cross-layer validation; by themselves they would likely undershoot the value of a full milestone unless tied to deeper platform work. 2 15 4

The clearest **speculative or too-large-for-one-milestone bets** are full monitoring/drift dashboards, dataset observability and contract evolution, and incremental cache invalidation. Those all point toward broader observability or metadata-management systems. They are not bad ideas, but they are a different magnitude of system design and are easier to justify after the repository has one more step of evidence discovery, audit linkage, and local exploration maturity. 10 27

Ranked milestone options

Strongest option: M35 – Artifact Lineage, Discovery, and Robustness-Aware Research Catalog. This is the best fit if you explicitly scope it as an **extension milestone over M29**, not a reintroduction of catalog/lineage from zero. It preserves identity, multiplies the value of M34 evidence, improves experiment lineage and artifact discovery, stays read-only on governance, and can support notebook/API/CLI/pipeline/CI surfaces without adding a new workflow engine. 10 13 16 18

Next option: M35 – Governance Audit Trail, Robustness Context, and Release Readiness. This is the best alternative if you want M35 to tilt more toward release-quality operational maturity than everyday research navigation. It is especially attractive if you want stronger linkage among promotion outcomes, robustness evidence, milestone validation bundles, and released artifacts without introducing any actual enforcement behavior. 16 18 8 9

Third option: M35 – Runtime Readiness Profiles and Reproducible Research Bundles. This is viable if your priority is production-adjacent hygiene, but it is weaker than the top two because StratLake still describes itself primarily as a research-review platform, and runtime/secrets/profile work moves the project closer to deployment infrastructure. It is a sensible later-phase milestone, but not the most natural immediate continuation of M34. 1 17

Preferred milestone package

The best practical recommendation is to keep your core direction, but sharpen the scope:

Final recommended M35 title: M35 – Artifact Lineage, Discovery, and Robustness-Aware Research Catalog

Preferred branch name: feature/m35-artifact-lineage-research-catalog

Recommended interpretation of that title: treat it as an **M29 extension milestone** that upgrades discovery, evidence linkage, and local review around already-existing catalog/query/lineage primitives, while promoting M34 robustness artifacts, M32 governance reports, and release-validation outputs to first-class searchable evidence surfaces. That keeps the platform deterministic, artifact-driven, read-only on governance, and fully compatible with notebook/API/CLI/pipeline/CI usage. 10 13 15 16 18

A practical M35 scope would ship the following issue breakdown:

- **Catalog schema and source expansion.** Make robustness bundles, governance bundles, milestone validation bundles, and release outputs first-class catalog/artifact record types, with deterministic status fields and portable paths.
- **Discovery and search facets.** Extend `query_catalog` and the CLI with robustness/governance/release filters such as `robustness_status`, `wfe_status`, `multiple_testing_status`, `promotion_status`, `review_status`, and validation/readiness presence.
- **Evidence lineage extensions.** Add lineage edges among strategy/alpha/portfolio/campaign runs and their robustness reports, governance reports, milestone bundles, and release validation artifacts, but only where explicit metadata exists.

- **Lightweight local explorer.** Add a read-only local explorer that can render stable table/JSON/Markdown and optionally static HTML over catalog results and evidence relationships. No server, no database, no canonical cache.
- **Notebook and API ergonomics.** Add thin helpers and script-backed examples that let notebooks discover relevant evidence without reconstructing artifact paths manually.
- **Validation and regression coverage.** Add focused tests for determinism, ordering, edge extraction, missing-evidence degradation, local render stability, and cross-platform path safety.
- **Docs and release package.** Add M35 architecture docs, user docs, one CI-safe example script, one notebook example, release notes, and a release validation checklist.

The validation strategy should stay inside the project's existing posture rather than inventing a new one: focused pytest slices for new catalog/discovery/explorer features, CI-safe example execution, docs/path portability checks, the existing cross-platform smoke matrix, milestone validation bundle compatibility, and regression coverage proving that governance remains observational and that promotion artifacts are unchanged by M35 features. [7](#) [8](#) [9](#) [26](#)

The explicit non-goals for the preferred M35 should be just as important as the goals: no new execution engine, no duplicate workflow logic, no persistent database/search backend, no policy replay, no policy enforcement, no feature-dataset contract migration system, no live monitoring service, and no weakening of M33/M34 portability and release guarantees. [10](#) [18](#) [17](#)

Interface fit for the preferred milestone is strong across the board: **notebook-friendly** because discovery helpers return in-memory records and portable artifact paths; **pipeline-friendly** because the layer is read-only over existing outputs; **API-friendly** because `src.catalog` already exposes a public package surface; **CLI-friendly** because query/discovery/explorer commands are naturally shellable; and **CI-friendly** because the examples can remain synthetic and local in the same style as current validation assets. [15](#) [3](#) [20](#) [19](#)

Codex kickoff prompt

```
Repository: christophermoverton/stratlake-trade-engine
Target branch: feature/m35-artifact-lineage-research-catalog
```

Objective:

Implement M35 as an extension of the existing M29 catalog/query/lineage stack, not as a new catalog from scratch.

Final milestone framing:

M35 – Artifact Lineage, Discovery, and Robustness-Aware Research Catalog

Primary goal:

Promote robustness, governance, milestone-validation, and release artifacts to first-class read-only discovery and lineage surfaces so researchers can find, compare, and inspect evidence across runs without manually navigating artifact roots.

Architecture constraints to preserve:

- one execution system, multiple entry points
- deterministic artifacts and deterministic ordering
- no duplicated workflow logic
- no new execution engine
- no new canonical registry
- no persistent database or search backend
- notebook/API/CLI/pipeline compatibility
- CI-safe examples
- portable path serialization only
- governance remains observable/read-only
- no silent policy replay or promotion decision changes
- no weakening of M33 cross-platform or release guarantees

Implementation scope:

1. Extend src.catalog models/indexer/query/lineage/validation so the following become first-class record families where artifacts exist:
 - M34 robustness bundles
 - M32 governance bundles
 - milestone validation bundles
 - release validation artifacts / release metadata surfaces already produced by workflows
2. Add deterministic query/filter support for evidence-oriented fields:
 - robustness_status
 - wfe_status
 - sample_size_status
 - sensitivity_status
 - multiple_testing_status
 - temporal_validation_status
 - promotion_status
 - review_status
 - readiness / validation presence
3. Add read-only lineage edges where explicit metadata exists:
 - run -> robustness report
 - run -> governance report
 - run -> milestone validation bundle references if available
 - release / validation linkage only if derivable without inventing new source-of-truth semantics
4. Add a lightweight local explorer:
 - CLI-first
 - deterministic table/json/markdown output
 - optional static HTML only if it stays local and testable
 - no server, no DB, no cache becoming canonical
5. Add notebook-friendly helpers and CI-safe examples using existing src.catalog and src.execution APIs only.
6. Add focused pytest coverage, docs/path portability coverage, and release-readiness docs.

Deliverables:

- source changes under src/catalog and minimal new CLI surfaces
- tests
- docs/M35 architecture doc
- docs/examples CI-safe script
- docs/examples notebook or script-backed notebook example
- README updates if warranted
- M35 release notes
- M35 release validation checklist

Important non-goals:

- do not create a new registry
- do not create a new workflow engine
- do not add policy enforcement
- do not add live monitoring services
- do not add dataset contract migration infrastructure in this milestone
- do not introduce absolute path leakage
- do not break current CLI/API/notebook/pipeline parity

Validation expectations:

- deterministic outputs and sorted keys
- empty-artifact-root and sparse-root graceful behavior
- cross-platform path portability
- CI-safe example with no external services or market-data requirements
- regressions proving canonical promotion artifacts are unchanged
- milestone validation and existing CI workflows remain green

1 README.md

<https://github.com/christophermoverton/stratlake-trade-engine/blob/main/README.md>

2 notebook_integration.md

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/docs/notebook_integration.md

3 pipeline_integration.md

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/docs/pipeline_integration.md

4 cross_layer_validation.md

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/docs/cross_layer_validation.md

5 __init__.py

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/src/execution/__init__.py

6 <https://github.com/christophermoverton/stratlake-trade-engine>

<https://github.com/christophermoverton/stratlake-trade-engine>

7 ci.yml

<https://github.com/christophermoverton/stratlake-trade-engine/blob/main/.github/workflows/ci.yml>

8 milestone_validation.yml

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/.github/workflows/milestone_validation.yml

9 **release.yml**

<https://github.com/christophermoverton/stratlake-trade-engine/blob/main/.github/workflows/release.yml>

10 **milestone_29_unified_research_catalog_design.md**

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/docs/milestone_29_unified_research_catalog_design.md

11 **catalog_indexer.md**

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/docs/catalog_indexer.md

12 **catalog_lineage.md**

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/docs/catalog_lineage.md

13 **catalog_query.md**

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/docs/catalog_query.md

14 **catalog_validation.md**

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/docs/catalog_validation.md

15 **catalog_notebook_workflows.md**

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/docs/catalog_notebook_workflows.md

16 **m32_governance_reporting_architecture.md**

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/docs/m32_governance_reporting_architecture.md

17 **cross_platform_reproducibility_audit.md**

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/docs/architecture/cross_platform_reproducibility_audit.md

18 **m34_statistical_robustness_architecture.md**

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/docs/m34_statistical_robustness_architecture.md

19 **m34_release_notes.md**

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/docs/m34_release_notes.md

20 **__init__.py**

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/src/catalog/__init__.py

21 **m31_release_notes.md**

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/docs/m31_release_notes.md

22 **<https://mlflow.org/docs/latest/ml/tracking/>**

<https://mlflow.org/docs/latest/ml/tracking/>

23 24 **<https://docs.evidentlyai.com/>**

<https://docs.evidentlyai.com/>

25 27 **<https://iceberg.apache.org/docs/latest/evolution/>**

<https://iceberg.apache.org/docs/latest/evolution/>

26 **m34_release_validation_checklist.md**

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/docs/m34_release_validation_checklist.md